

NYU Tandon School of Engineering

Assignment: Term Project

Course: ECE-GY 6263 Game Theory

Instructor: Prof. Quanyan Zhu

Fall 2025

Student: Kevin Ho

Project title:

Optimal Control and Differential Game Theory

ABSTRACT

Optimization is about choosing the best choice or most optimal that gives the least cost or best reward, this relates very much with game theory. In game theory, there may be one or more than one player where each of them have their own objectives and their own strategies to meet that objective while competing with each other. We want to introduce some definitions and review zero-sum games. The games that are mainly focused on, are non-cooperative games. The saddle-point and equilibrium pairs will lead maximin and minimax strategies, where it is useful to reach a pure-Nash equilibrium for a zero-sum game. Some game dynamics will be introduced and examined, such as information structure and continuous time differential games. Discrete-time and continuous time dynamic programming will be used to find optimal policies in the case of open-loop and state-feedback for one player dynamic or differential games. Topics such as linear quadratic dynamic games will be briefly mentioned due to its correlation with state-space representation in discrete-time and continuous-time.

INTRODUCTION

What are *players*? The players are the agents that make decisions in the games. The *rules* of the game govern the players and their effects. The *information structure* defines what each player knows about the game before making each decision. The goals of each player are known to be *objectives*.

The player(s) must start the game with the initial position (start point). Other positions may also occur in the game. At each position, the player (or players) makes a move or an *action* from that position and is capable of moving to another position next turn. For each game position P , there must be at least one sequence of moves from the initial position to P (the game needs a move from the start point). Some positions allow terminal positions, positions that allow the game to be *game over* (end point). So in the game, there should be a sequence of moves that goes from the start point and to an end point. If the rules of the game make it so that an infinite sequence of moves are not possible, then the game is *finite*.

Now regarding about the players, we need to make some assumption about each player's rationality, and asking questions such as:

- Will the players always pursue their best interests to fulfill their objectives? [Yes]
- Will the players form coalitions? [No]
- Will the players trust each other? [No]

If you are able to answer the questions and give the same answer as is in the brackets, then you have a *noncooperative game* [2].

Every terminal position results in a payoff to each of the players (payoff can also occur before the game ends). Often, payoff numbers (e.g. amounts of money) are the amounts that the various players win or lose at that terminal position (positive numbers = winnings; negative numbers = losses). If the total sum of the payoffs (at each terminal position) is zero, then we call this game the *zero-sum* game. Note, a zero-sum game is a subset of a noncooperative game because players have opposite objectives.

Sometimes, you just simply want to indicate whether the player lost or won by indicating -1 or +1, respectively. If ties or draw, then indicate as 0. Alternatively, a win can be W, a loss as L, and a draw as D.

A very basic game can be pictured by a tree diagram, where the nodes labeled which player(s) must move at that position and the branches (or branch) coming out of each node will be the possible moves (or move) to another position. At each terminal node (terminal position) the payoffs of each of them are shown as a *n-tuple*.

If a position that has various possible moves (each of them have a probability) is called a *random position* and a move act on that random position to get to another position is called *random move*. A game without random positions is known to be *deterministic*, where the game depends purely on each player's decision.

Also, some games may have outcomes of certain moves that may not be known to all of the players. However, when the players know the outcomes of every possible move, this game is known to be a *game with perfect information*. A decision rule that a player uses to select actions that are based on available information, are called a *policy*. Note, policy in some textbooks is referred to as *strategy* [2].

In a game with numerical payoffs, a strategy is *non-losing* if the players follow moves that always achieve non-negative payoffs, disregarding the moves of other player(s). Also, a *winning strategy* always leads to positive payoffs, disregarding the moves of other player(s). A *win-or-draw strategy* will always result in a Win or a Draw, disregarding the moves of other player(s).

COMBINATORIAL GAME

A game is a *combinatorial game* if it satisfies the following:

1. There are only 2 players.
2. It is deterministic (no random moves).
3. It is finite (no infinite sequence of moves).
4. It is a game with perfect information.
5. It is a zero-sum game (payoffs sum off to be zero).

In any combinatorial game, at least one of the players has a non-losing strategy, meaning a player can win or draw. A quick proof can be that player A can make any move from position P1 to P2, that is not a non-losing position for A. Also, there should be at least one possible move M leading from P1 to P2, that is not a non-losing position for player B. P1 cannot be a non-losing position. Since moving M leading from P1 to P2 doesn't lead to a non-losing position, we can try moving P2 to P3, which is also a non-losing position for B. We can keep moving from position P3 to P4, then P4 to P5, and so on. This is an infinite sequence of moves which is contradicting, since combinatorial games is a finite game. This shows that in any combinatorial game, at least one of the players has a non-losing strategy and this is also known as the fundamental theorem. (see [3] for full proof)

In a combinatorial game, if one player X has a non-losing strategy, but has no winning strategy, then the other player Y must have a non-losing strategy [3]. That means, either players have a winning strategy, or both players have non-losing strategies. If a draw is impossible in a game, then one of the players must have a winning strategy. Note, combinational game can be categorized as noncooperative game and zero-sum games, but with extra constraints (perfect info., deterministic, alternating moves, 2 players)

TWO-PERSON ZERO-SUM GAMES

Considered a game where player A has strategies $\{A_1, \dots, A_m\}$ and that player B has strategies $\{B_1, \dots, B_n\}$. This game is a zero-sum game, where player A has a payoff $P(A_i, B_j)$ for the strategies (A_i, B_j) ; player B has a corresponding payoff $-P(A_i, B_j)$, since $P(A_i, B_j) - P(A_i, B_j) = 0$. This game is in *normal form* because payoffs can be described as a payoff matrix A (where the each entries are in i^{th} row and j^{th} column), list of available strategies for each player, and a list of players. Also, this matrix A is size $m \times n$ with elements of real numbers, where the strategies of player A are listed in the rows and the strategies of player B are listed in the columns. This means that each players have a finite set of actions or *action space*:

- Player A has m actions: $\{1, 2, \dots, m\}$
- Player B has n actions: $\{1, 2, \dots, n\}$

The outcome J or payoff is entry a_{ij} of the matrix that when Player A selects i^{th} action and Player B selects j^{th} action.

To demonstrate a zero-sum game in physical point of view, considered a rope pulling game from [2],

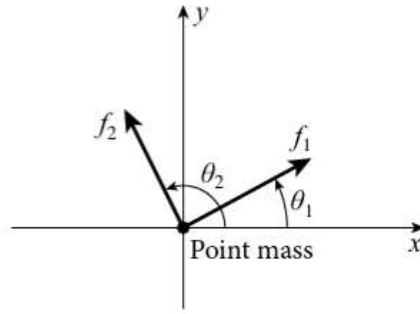


Figure 1: Rope-pulling game from [2]

The **rule of the game**: 2 players pull a point mass by exerting force $f_1 = \sqrt{(f_1^{(x)})^2 + (f_1^{(y)})^2}$ and $f_2 = \sqrt{(f_2^{(x)})^2 + (f_2^{(y)})^2}$. Both players exert forces with the same magnitude ($|f_1| = |f_2|$), but they pull in different directions $\theta_1(t)$ and $\theta_2(t)$, respectively.

The game is played for 1 second. We assume the mass is initially at rest, unit forces ($|f|=1$ N) and a unit mass ($m=1$ kg). So according to Newton's law ($f=ma$), as shown in the (Fig 1) point mass moves according to

$$\ddot{x} = \cos(\theta_1(t)) + \cos(\theta_2(t)), \quad \dot{x}(0) = 0, \quad x(0) = 0 \quad (1)$$

$$\ddot{y} = \sin(\theta_1(t)) + \sin(\theta_2(t)), \quad \dot{y}(0) = 0, \quad y(0) = 0 \quad (2)$$

due to the total horizontal force being $f^{(x)} = f_1^{(x)} + f_2^{(x)}$ and total vertical force being $f^{(y)} = f_1^{(y)} + f_2^{(y)}$. Note, $\theta_1(t)$ and $\theta_2(t)$ depends on the decision of the players. Now we want to demonstrate a zero-sum game. Assume Player 1 want to *maximize* $x(1)$, whereas Player 2 wants to minimize $x(1)$. Now, we will claim that the optimal solution for this game is given by

$$\textbf{Player 1: } \theta_1(t) = 0, \quad \forall t \in [0, 1], \textbf{Player 2: } \theta_2(t) = \pi, \quad \forall t \in [0, 1], \quad (3)$$

which results in no motion ($\ddot{x} = \ddot{y} = 0$) because f_1 and f_2 canceled each other out (cf. Fig 2), leading to $x(1) = y(1) = 0$.

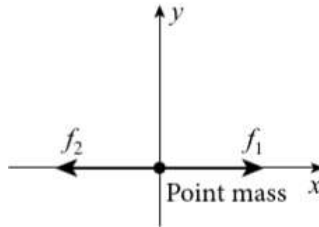


Figure 2: Rope-pulling game for zero-sum from [2]

SADDLE POINTS AND EQUILIBRIUM PAIRS

A *saddle point* of a matrix is the entry that maximizes the row of the matrix and minimizes the columns of the matrix. In other words, one player is minimizing and another player is maximizing. This means that there is an *equilibrium pair*¹ or a pair (A_i^*, B_j^*) of strategies that lead to the corresponding payoff, which results in a saddle point. An equilibrium pair is stable such that deviating to another pair of strategies is considered to be inadvisable. For example, if a player X plays his strategy in that equilibrium pair, but the other player Y chooses not to play in that pair, then the player Y will get a poorer payoff compared if he were to play in the equilibrium pair. A saddle point that does not exist if a game is unstable, meaning whichever strategies the players choose, either one or both of the players would gain regardless of which strategies chosen. So, if a game has a

¹ The symbol " * " mean optimal (or equilibrium) solution

saddle point, then the game is called a *strictly determined game*. If a game can be represented as a matrix, then it is called a *matrix game*. A matrix game with a saddle point(s) has two properties:

1. If a matrix game has saddle points, then all entries at these saddle points are equal.
2. If (A_i, B_j) and (A_r, B_s) are saddle points, so are (A_i, B_s) and (A_r, B_j) .

For example, consider the following table:

	Player 2's strategy B_1	Player 2's strategy B_2
Player 1's strategy A_1	2	3
Player 1's strategy A_2	1	4

The strategies of Player 1 are listed in the rows (highlighted in green) and the strategies of Player 2 are listed in the columns (highlighted in red). Both players can choose any of their own 2 strategies. The numerical entries can represent the outcomes (punishment or rewards) depending on the strategies chosen by the players, such as Player 1 can get paid \$1 if the pair strategy (A_2, B_1) is chosen by both players. Note in zero-sum games, one player's gain is exactly the other's loss and a single payoff matrix is often representing the row player's payoff. Now the table can be redrawn as a two-person zero-sum game in normal form with the 2×2 :

$$\begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix}$$

Now, we can try to find the saddle point, we can assume that Player 1 wants to maximize and Player 2 wants to minimize. So we can first maximize the columns: $\max\{2, 1\} = 2$ and $\max\{3, 4\} = 4$, giving us $\{2, 4\}$ or $[2, 4]$, which is what Player B would anticipate from Player A depending on which "column"² that Player B decides to choose. Next minimize the row: $\min\{\max\{2, 1\}, \max\{3, 4\}\} = \min\{2, 4\} = 2$ and with strategy pair (A_1, B_1) . Player B clearly prefers giving \$2 instead of \$4.

Note, if you minimize rows first and then maximizes column, we will still result with 2 and strategy pair (A_1, B_1) ;

$\max\{\min\{2, 3\}, \min\{1, 4\}\} = 2$. Also, $\max\{\min\{2, 3\}, \min\{1, 4\}\} = \min\{\max\{2, 1\}, \max\{3, 4\}\} = 2$, telling us that 2 is a saddle point and equilibrium pair (A_1, B_1) . In a zero-sum game, saddle point is also referred to as *pure-strategy Nash*

equilibrium because none of the players are not willing to deviate³ from the pair strategy or equilibrium pair (A_i^*, B_j^*) , and this is the best response against each other, which will always be 100% chosen as the strategy (this is the reason why this is called *pure strategy*). However, whichever player is minimizing or maximizing does matter. If Player 1 wants to minimize and Player 2 wants to maximize, the saddle point will be 3 with equilibrium pair (A_1, B_2) .

Consider another example, we are given the following matrix A:

$$A = \begin{bmatrix} 4 & 1 & 2 & 1 \\ 2 & 0 & 1 & -1 \\ 2 & 1 & 3 & 1 \\ 0 & 0 & 5 & 0 \end{bmatrix}$$

Again, the strategies of Player 1 are listed in the rows and the strategies of Player 2 are listed in the columns. This matrix happens to have 4 saddle points (Player 1 is maximizing and Player 2 is minimizing), and each can be found by

² This may be confusing at first, and one may ask "why are we maximizing the column and not the row?" We can think it as Player A is looking for the best worst case scenario because he is anticipating that Player B will give him the worst case and Player A would have to pick the best case out of the worst possible case. Similarly, we can think of Player B looking for the worst best case scenario and here we are actually looking at Player B's point of view because he knows that Player A will pick the best cases, so Player B would have to pick the worst case out of the best cases.

³ One way to think about whether players are not willing to deviate, we can first consider Player A's point of view. Player A obviously wants the pay to be highest because he is a maximizer, and choosing row 2 and result with a payoff of 4 would be difficult because Player B can choose column 1. This results in Player A to choose row 1 since whether Player B chooses column 1 or 2, the payoff will always be between 2 and 3, making it less risky. Of course, Player B knows that Player A would pick whatever is highest, then Player B wouldn't choose column 2 because Player A would get a payoff of either 3 or 4. Thus, Player B would choose column 1, since Player A would choose either payoff to be 1 or 2. All this shows that the equilibrium point is at row 1 and column 1 because none of the players are willing to deviate from their strategies.

$\max\{\min\{4, 1, 2, 1\}, \min\{2, 0, 1, -1\}, \min\{2, 1, 3, 1\}, \min\{0, 0, 5, 0\}\} = \max\{1, -1, 1, 0\} = 1$. This also means we have 4 equilibrium pairs at (A_1, B_2) , (A_3, B_2) , (A_1, B_4) , and (A_3, B_4) . Here, saddle-point equilibria exist because there is more than one equilibrium.

When the matrix game is in normal form and has saddle points, then the common value V of the entries at all saddle points will be called the *value* of the game [3]. The value V is the payoff for player A (payoff is $-V$ for player B) when both players play in the equilibrium pair resulting in a saddle point. Now, if the value $V = 0$, then the game is *fair* since neither player has advantage over the other since they both chose their strategies correctly.

CONSTANT-SUM GAMES

If the total sum of the payoffs (at each terminal position) is not zero but a constant K , then we call this game the *constant-sum* game. A constant-sum game can essentially be converted to a zero-sum game.

For another example, considered 2×2 payoff bimatrix A for two-person game:

$$A = \begin{bmatrix} (7, 3) & (2, 8) \\ (6, 4) & (1, 9) \end{bmatrix} \rightarrow A' = \begin{bmatrix} (2, -2) & (-3, 3) \\ (1, -1) & (-4, 4) \end{bmatrix}$$

Each outcome in the matrix has two numbers (player 1 pay off, player 2 payoff). Here the sum of payoff is always $K = 10$ (the sum of each payoff in the matrix is A 10), making this a 10-sum game. The 2×2 payoff matrix A turn into a zero-sum game by subtracting 5 to each outcome, resulting A' . You can see that the sum of each payoff in the matrix A' is 0; e.g. $\text{sum}\{+2, -2\} = 0$. Generally, the new payoff for each player is the old payoff minus $K/2$, making the constant-sum game into a zero-sum game. If the sum of each payoff in the matrix is not all 10, then this is a *non-zero-sum* game.

MAXIMIN AND MINIMAX

Another way to look at the payoff matrix of a game in normal form is with *maximin* and *minimax* strategies⁴. For example, we have a payoff matrix A (where the each entries are in i^{th} row and j^{th} column) and let a_{ij} be denoted as the entries. Again we have two players, Player A and Player B, and choose rows and columns, respectively. Also, Player A is maximizing and Player B is minimizing.

Now consider a case where the saddle point doesn't exist. Player A wants to deal with the *best worst case*. In other words, (after Player A makes a move) Player B will pick the column that gives Player A the worst (smallest) payoffs, which is the minimum entries of each corresponding row and Player A will be left with choosing the i^{th} row from m numbers of rows that gives the best (biggest) payoff out of all the worst possible payoffs. To demonstrate this, consider the following matrix A ,

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} = \begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix}.$$

Player A (row player) is now going to anticipate the worst case by minimizing each rows; $\min_{j=1,2} \{3, 2\} = 2$ and

$\min_{j=1,2} \{1, 4\} = 1$. Note, we get a column vector $[2, 1]^T$. Now Player A is going to choose the best of the worst cases by maximizing; $\max_{i=1,2} \{\min\{3, 2\}, \min\{1, 4\}\} = 2$. This means that if Player A chooses row 1, Player B can push the payoff down to 2 by selecting column 2; vice versa, if Player A chooses row 2, Player B can push the payoff down to 1 by selecting column 1. Then seeing that row 1 is the best of the worst outcome, Player A will choose strategy A_1 . Such a strategy A_i (there is at least one such) is called a *maximin strategy* for Player A. All of this can be generalized as $\max_i \min_j a_{ij}$ or as *maximin*, which will give the *lower game value* $\underline{V}$.

Similarly, Player B is trying to look for the worst best case or maximize the amount that he is allowed to lose (Player B is not planning to lose more than he can). In other words, (after Player B makes a move) Player A will pick the row that makes

⁴ For Player A, the maximin strategy is sometimes referred to as *security policy*. Since Player B is the opponent of Player A, minimax strategy will be referred to as the *opponent's security policy*.

Player B pays the most payoff, which is the maximum entries of each corresponding column and Player B will be left with choosing the j^{th} column from n numbers of columns that give the worst payoff out of all the best possible payoffs for Player A. To demonstrate this, again consider the previous matrix game, Player B will maximize each column and would be left with a row vector to be minimized. So considered matrix A again, Player B will maximize each column; $\max_{i=1,2} \{3, 1\} = 3$ and $\max_{i=1,2} \{2, 4\} = 4$. Note, we get a row of elements $\{3, 4\}$ that Player A may choose from Player B. And then, Player B is going to choose the worst of the best cases for Player A by minimizing; $\min_{j=1,2} \{\max\{3, 1\}, \max\{2, 4\}\} = 3$. This means that if Player B chooses column 1, Player A can push the payoff up to 3 by selecting row 1; vice versa, if Player B chooses column 2, Player A can push the payoff up to 4 by selecting row 2. Then seeing that column 1 is the worst of the best outcome for Player A, Player B will choose strategy B_1 . Essentially Player B is looking for the *least of the worst losses*. Such a strategy B_j (there is at least one such) is called a *minimax strategy* for Player B. All of this can be generalized as $\min_j \max_i a_{ij}$ or as *minimax*, which will give the *upper game value* $\bar{V}$.

From our example, we can intuitively see that $\maximin \leq \minimax$ for any matrix, to prove this, consider a matrix $m \times n$ size. According to [3], if u_i is the minimum in the i^{th} row and w_j is the maximum in the j^{th} column, and if a_{ij} is the entry in the i^{th} row and j^{th} column, then $u_i \leq a_{ij} \leq w_j$. This shows that (looking at the right-hand-side inequality) minimax can be $a_{ij} \leq w_j$ and this means that it can be less than or equal to every column maximum. And, this also shows that every row minimum is less than or equal to the minimax and, therefore, $\maximin \leq \minimax$.

Another thing is that when $\maximin = \minimax$ occurs for any matrix, if and only if the matrix has a saddle point. This means that $u_i = a_{ij} = w_j$ which makes a_{ij} the saddle point. The proof can be found in [3]. Now, for a matrix game defined by matrix A , the *security level* for Player A can be mathematically expressed as,

$$\underline{V}(A) := \max_{i \in \{1, 2, \dots, m\}} \min_{j \in \{1, 2, \dots, n\}} a_{ij} \quad (4)$$

and the corresponding security policy for Player A is any i^* that achieves security level (achieves the maximum in this case);

$$\min_j a_{i^*j} = \underline{V}(A) \quad (5)$$

Also, the *security level* for Player B can be mathematically expressed as,

$$\bar{V}(A) := \min_{j \in \{1, 2, \dots, n\}} \max_{i \in \{1, 2, \dots, m\}} a_{ij} \quad (6)$$

and the corresponding security policy for Player B is any j^* that achieves security level (achieves the minimum in this case);

$$\max_i a_{ij^*} = \bar{V}(A) \quad (7)$$

Then the inequality, $\maximin \leq \minimax$ can be expressed as

$$\underline{V}(A) \leq \bar{V}(A). \quad (8)$$

Also, for the case of saddle point can be expressed as

$$\underline{V}(A) = \bar{V}(A). \quad (9)$$

Note, if Player A is minimizing and Player B is maximizing, then we will minimize with respect to i and maximize with respect to j , respectively. Also, the inequality, $\maximin \leq \minimax$ also holds.

When there is a saddle point, the safest strategies for Player A and Player B are maximin strategy and minimax strategy, respectively. However, when a matrix game has no saddle point, the strategies will cause the players to change their strategies every round. To demonstrate this, considered matrix A again. Recalled, there is no saddle point. The maximin is 2 for strategy A_1 and the minimax is 3 for strategy B_1 . We can start by assuming that Player A and Player B played their pair strategies

(A_1, B_1) . The payoff for Player A is 3 (-3 for Player B or can think of it as Player B losing \$3)⁵, but Player B clearly preferred giving payoff as 2 instead of 3. Player B notice that he can give a lesser payoff if he chooses strategy B_2 . Then, in the next game, the strategies will be (A_1, B_2) and giving the Player A a payoff of 2. Now, Player A obviously will switch to strategy A_2 since it will give Player A even more payoff. So, in the next game, the strategies will be (A_2, B_2) and giving the Player A a payoff of 4. Then for next game, the strategies will be (A_2, B_1) , giving a payoff of 1. One can see that the strategies will then go back in a cycle, going back to the beginning with the strategies (A_1, B_1) . So, the strategies will keep repeating in this way endlessly. This type of game with no saddle points requires *mixed strategies*, but since that is not our main focus, we won't be going over it.

Notice that we have been assuming that Player A and Player B are rational, meaning they know the payoff matrix and have a logical mindset. Also, we've been assuming that the players want to maximize their gains and minimize their losses.

DOMINANCE

Sometimes you want to *eliminate* strategies and keep a strategy that is the best or dominates over other strategies. Meaning, (assume you are the row player or Player A who is maximizing) if each entry of one **row X** of the matrix of a game is greater or equal to the entry in the **same column** of another **row Y**, then one can say “row X *dominates* row Y.” In other words, Player A will have the common sense to never use the strategy of row Y and would choose the strategy of row X, since it offers an equal or better payoff. Resulting with row Y to be unnecessary, thus analysis of the game is simplified if row Y is eliminated.

Similarly for the column player or Player B who is minimizing, if each entry of one **column V** of the matrix of a game is greater or equal to the entry in the **same row** of another **column W**, then one can say “column W *dominates* column V.” In other words, Player B will have the common sense to never use the strategy of column V and would choose the strategy of column W, since it offers an equal or lesser gain for Player A (but is equal or better gain for Player B). Resulting with column V to be unnecessary, thus analysis of the game is simplified if column V is eliminated.

Eliminations of rows or columns by means of dominance relation often helps in the search for the best strategies for the players [3]. For example, from [3], consider the 3×3 matrix on the left below.

$$\begin{bmatrix} -4 & 2 & -3 \\ -2 & -5 & 3 \\ -1 & 0 & 1 \end{bmatrix} \rightarrow \begin{bmatrix} -4 & 2 \\ -2 & -5 \\ -1 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} -4 & 2 \\ -1 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} -4 \\ -1 \end{bmatrix} \rightarrow [-1]$$

Looking at the first and third columns, we see that the 1st column dominates the 3rd column because every value in the 1st column is less than the value in the 3rd column. Then we proceed with eliminating the 3rd column, and get a 3×2 matrix. Now, the 3rd row dominates the 2nd row because every value in the 3rd row is greater than the value in the 2nd row. Then we proceed with eliminating the 2nd row, and get a 2×2 matrix. Currently, the 1st column dominates the 2nd column, and we get a 2×1 matrix by eliminating the 2nd column. Finally, we see that the 2nd row dominates the 1st row and we result with a 1×1 matrix. Notice, this is the saddle point of the original matrix if Player A is maximizing and Player B is minimizing.

Note, if Player A is minimizing and if each entry of one **row X** of the matrix of a game is lesser or equal to the entry in the **same column** of another **row Y**, then one can say “row X *dominates* row Y.” This can be mathematically expressed as

$$a_{xj} \leq a_{yj} \quad \forall jth \text{ columns} \quad (10)$$

Similarly, if Player B is maximizing and if each entry of one **column V** of the matrix of a game is lesser or equal to the entry in the **same row** of another **column W**, then one can say “column W *dominates* column V.” This can be mathematically expressed as

$$a_{iw} \geq a_{iv} \quad \forall ith \text{ rows} \quad (11)$$

⁵ Player B wants to minimize the Player A's gain or x, and can think of it as $\min\{x\}$ and we can mathematically express it as $\max\{-x\}$, due to the minimax theorem. In some textbooks, both players are maximizing which can be confusing when dealing with a single matrix instead of bimatrix.

Also, dominating strategy is useful in reducing a large game to a smaller reduced game. For example, consider this matrix on the left below from [2]:

$$\begin{bmatrix} 3 & -1 & 0 & -1 \\ 4 & 1 & 2 & 0 \\ -1 & 0 & 1 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 3 & -1 & 0 & -1 \\ -1 & 0 & 1 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 3 & 0 \\ -1 & 1 \end{bmatrix}$$

Here we are assuming the row player is minimizing and the column player is maximizing. Looking at the 1st and 2nd row, we see that the 1st row dominates the 2nd row because every value in the 1st row is less than the values in the 2nd row. Then we proceed with eliminating the 2nd row, and get a 2×4 matrix. Now, the 3rd column dominates the 2nd and 4th columns because every value in the 3rd column is greater than values in the 2nd and 4th columns. Finally, eliminating the 2nd and 4th columns, we ended up with a 2×2 matrix.

One may think that searching for the dominance relationship will lead to the saddle point, however that is not always true. For example, from [3], considered the matrix,

$$\begin{bmatrix} 3 & 5 & 8 \\ 2 & 1 & 9 \\ 1 & 6 & 0 \end{bmatrix}$$

and it has a saddle point 3. However, here you cannot find the saddle point using the dominance relationship, in other words, there are no dominance relations.

NASH EQUILIBRIUM

In zero-sum games, Nash equilibrium (NE) solutions are also called *saddle-point solutions*. To understand the definition of NE, considered a game with two players P_1 and P_2 . A pair of strategies or policies (π_1, π_2) is said to be NE if the following two conditions hold:

C1. If P_1 uses policy π_1 , then there is admissible policy for P_2 that does strictly better than π_2 .

C2. If P_2 uses policy π_2 , then there is admissible policy for P_1 that does strictly better than π_1 .

Note, C1 and C2 are referred to as *Nash equilibrium conditions*. By the way, C1 does not need π_2 to be strictly better than all the other policies, just no worse. Similarly for π_1 in C2. As mentioned before, P_1 and P_2 start playing at NE (π_1, π_2) and none of the players gain more by deviating from these policies, making NE *stable*.

COMPUTATIONAL COMPLEXITY

Sometime, one might want to consider a large scale system and determine the number of operations needed to find a security policy. Suppose first, we want to maximize a function $f(i)$ defined over a discrete set $\{1, 2, \dots, n\}$:

i	1	2	3	...	n
$f(i)$	$f(1)$	$f(2)$	$f(3)$	...	$f(n)$

The number of operations to find the maximum of the function f is $n - 1$; start by comparing $f(1)$ and $f(2)$, then the larger result compares with $f(3)$, and so on.

Suppose now, instead of a maximizing function, we want to find the security policies from an $m \times n$ matrix

$$A = \begin{bmatrix} & \vdots & \\ \dots & a_{ij} & \dots \\ & \vdots & \end{bmatrix}.$$

Assume that P_1 is the minimizer and P_2 is the maximizer. To find a security policy for P_1 , one need to considered:

1. Each row of the matrix can be viewed as a function with a discrete set. For each row, the number of operations to find the maximum of the function f is $n - 1$. Since there are m numbers of rows, the total number of operations is $m(n - 1)$.
2. After maximizing each row, you would have to minimize the resulting column vector. Each column of the matrix can be viewed as a function with a discrete set. For each column, the number of operations to find the minimum of the function f is $m - 1$.

Then the total amount of operations to find a security policy for P_1 is $m(n - 1) + m - 1 = mn - 1$, by the way, that is also the total amount of operations needed to find a security policy for P_2 .

Now given a candidate saddle point, you want verify if it is legit using eq. (5) and (7), then you need $m - 1 + n - 1$ or $m + n - 2$ amount of comparison operators.

GENERAL ZERO-SUM GAMES

Now, we want to generalize zero-sum games. Consider a general two-player zero-sum game G in which the players P_1 and P_2 are allowed to select policies within action spaces Γ_1 and Γ_2 , respectively. Generally, action spaces can be finite or infinite as in a matrix game with pure policies or mixed policies, respectively. Previously, the payoff is denoted as $P(A_i, B_j)$ for the pair strategies (A_i, B_j) . Here, from now on, the *outcome of the game* will be denoted by $J(\gamma, \sigma)$ for a pair of policies (γ, σ) , where $\gamma \in \Gamma_1$ and $\sigma \in \Gamma_2$. In most textbooks, it is often P_1 is the minimizer and P_2 is the maximizer. Meaning, P_1 want to minimize the cost $J(\gamma, \sigma)$ and P_2 wants to maximize the gain from it. The security level for P_1 is defined by

$$\bar{V}_{\Gamma_1, \Gamma_2}(G) := \inf_{\gamma \in \Gamma_1} \sup_{\sigma \in \Gamma_2} J(\gamma, \sigma) \quad (12)$$

where this can be thought as P_1 is looking for the *least of the worst losses* because $\sup_{\sigma \in \Gamma_2}$ will give losses caused by P_2 (from P_1 's perspective) and $\inf_{\gamma \in \Gamma_1}$ will give the least of the losses. The corresponding security policy for P_1 is any policy $\gamma^* \in \Gamma_1$ for which infimum above achieved,

$$\sup_{\sigma \in \Gamma_2} J(\gamma^*, \sigma) = \bar{V}_{\Gamma_1, \Gamma_2}(G) := \inf_{\gamma \in \Gamma_1} \sup_{\sigma \in \Gamma_2} J(\gamma, \sigma) \quad (13)$$

Also, the security level for P_2 is defined by

$$\underline{V}_{\Gamma_1, \Gamma_2}(G) := \sup_{\sigma \in \Gamma_2} \inf_{\gamma \in \Gamma_1} J(\gamma, \sigma) \quad (14)$$

where this can be though as P_2 is looking for *best worst case* because $\inf_{\gamma \in \Gamma_1}$ will give the worst payoffs/outcomes given by P_1 (from P_2 's perspective) and $\sup_{\sigma \in \Gamma_2}$ will give the best of the worst possible payoffs. The corresponding security policy for P_2 is any policy $\sigma^* \in \Gamma_2$ for which the supremum above is achieve,

$$\inf_{\gamma \in \Gamma_1} J(\gamma, \sigma^*) = \underline{V}_{\Gamma_1, \Gamma_2}(G) := \sup_{\sigma \in \Gamma_2} \inf_{\gamma \in \Gamma_1} J(\gamma, \sigma) \quad (15)$$

Then a pair of policies $(\gamma^*, \sigma^*) \in \Gamma_1 \times \Gamma_2$ is called a *saddle-point equilibrium* (SPE) if

$$J(\gamma^*, \sigma) \leq J(\gamma^*, \sigma^*) \leq J(\gamma, \sigma^*), \quad \forall \gamma \in \Gamma_1, \sigma \in \Gamma_2. \quad (16)$$

and $J(\gamma^*, \sigma^*)$ is called the *saddle-point value*.

Infimum is the number y^* that is the highest lower bound of a set $S \subset \mathbb{R}$,

$$y^* \leq s, \quad \forall s \in S. \quad (17)$$

Note, when there is no number y^* that satisfies eq. (17), the infimum is defined to be $-\infty$. Also, the infimum may not belong to the set. However, if it does belong to the set, then infimum is a *minimum*. For example from [2],

$$\inf_{x \geq 0} e^{-x} = 0, \quad \inf_{x \leq 0} e^{-x} = \min_{x \leq 0} e^{-x} = 1.$$

Notice that the infimum of the set in the left-hand-side is not a minimum since 0 does not belong to the set, but the infimum of the set in the right-hand-side does belong to the set.

Supremum is the number z^* that is the lowest upper bound of a set $S \subset \mathbb{R}$,

$$z^* \geq s, \quad \forall s \in S. \quad (18)$$

Note, when there is no number z^* that satisfies eq. (18), the infimum is defined to be $+\infty$. Also, every non-zero set has a supremum. When the supremum belongs to the set, then supremum is a *maximum*.

The following properties hold for every zero-sum game:

- Security levels are well defined and unique. This may include infinite values like $+\infty$ or $-\infty$, and security policies may not exist.
- The security levels always satisfy the following inequality:

$$\underline{V}_{\Gamma_1, \Gamma_2}(G) := \sup_{\sigma \in \Gamma_2} \inf_{\gamma \in \Gamma_1} J(\gamma, \sigma) \leq \inf_{\gamma \in \Gamma_1} \sup_{\sigma \in \Gamma_2} J(\gamma, \sigma) =: \bar{V}_{\Gamma_1, \Gamma_2}(G)$$

- When $\underline{V}_{\Gamma_1, \Gamma_2}(G) = \bar{V}_{\Gamma_1, \Gamma_2}(G)$ and there exist security policies γ^* and σ^* for P_1 and P_2 , respectively then (γ^*, σ^*) is a SPE and its value $J(\gamma^*, \sigma^*)$ equal to $\underline{V}_{\Gamma_1, \Gamma_2}(G) = \bar{V}_{\Gamma_1, \Gamma_2}(G)$.
- If there exist a SPE (γ^*, σ^*) then γ^* and σ^* are security policies for P_1 and P_2 , respectively $\underline{V}_{\Gamma_1, \Gamma_2}(G) = \bar{V}_{\Gamma_1, \Gamma_2}(G) = J(\gamma^*, \sigma^*)$.
- If (γ_1^*, σ_1^*) and (γ_2^*, σ_2^*) are both SPE then (γ_1^*, σ_2^*) and (γ_2^*, σ_1^*) are also SPE, and all these equilibria have exactly the same value.

GAME DYNAMICS

So far we have focused on *single-stage games* or *normal-form games* in which we did minimax and maximin strategies, where each player must choose an action only once per game. Now, a game that consists of a sequence of rounds (or stages) is called a *Multi-stage game*, and in each round the players have the opportunity to select one action.

Consider a two-player multi-stage game and the following notation: For each stage $k \in \{1, 2, \dots, K\}$,

1. x_k denotes the state at k th stage,
2. u_k denotes the action of player P_1 at the k th stage,
3. d_k denotes the action of player P_2 at the k th stage.

In this case, the *state evolution* can be mathematically described by:

$$x_{k+1} = f_k(x_k, u_k, d_k), \quad \forall k \in \{1, 2, \dots, K-1\}, \quad (19)$$

where,

- x_{k+1} is the state at stage $k+1$,
- $f_k(\cdot)$ is a dynamic function f that gives the *dynamic* at stage k ,

and this can generalize:

- games described by graphs or tree diagrams,
- games with infinitely many stages ($K = \infty$),
- or games with action spaces that are not finite sets.

(In some textbooks, $k \in \{0, 1, \dots, N-1\}$, where x_0 is the initial state) Generally, if the evolution of a game is represented by an equation such as eq. (19), then the game is called a *dynamic game* and the eq. (19) is often called *dynamics of the game*. The set X where the state x_k takes values is called the *state-space* of the game; if the state is $x \in \mathbb{R}^n$, then the state space is $X \subseteq \mathbb{R}^n$.

The outcome J_i for a particular player P_i , $i \in \{1, 2\}$ in a multi-stage game in form of a tree diagram is a function of the state of the game at the last stage K and the actions taken by the players at this stage:

$$J_i(x_K, u_K, d_K). \quad (20)$$

However, when the game is described by a graph that is not a tree, the outcome may be different depending on the game. In this case, the outcome J_i may depend on all the decision made by both players from the start of the game:

$$J_i(u_1, d_1, u_2, d_2, \dots, u_K, d_K).$$

A dynamic game is said to have a *stage-additive cost* when the outcome J_i to be *minimized* (minimized the losses, cost, penalty, or error) can be written as

$$\sum_{k=1}^K g_k^i(x_k, u_k, d_k), \quad (21)$$

where,

- $g_k^i(x_k, u_k, d_k)$ is the *stage cost* or a *single-step cost* at one stage k .

When all g_k^i are equal to zero, except for the last g_K^i , the game is said to have a *terminal cost*.

A. Information structures

Dynamic games can have a wide range of information structures, but we will mostly focus on two structures: open loop and (perfect) state feedback.

In *open-loop (OL)* dynamic games, players do not gain any information as the game is played (other than the current stage) and must make their decision mainly based on a priori information. For an open-loop dynamic game, in the [2], the policies as functions of the initial state x_1 :

when player P_1 uses an open-loop policy $\gamma^{OL} := \{\gamma_1^{OL}, \gamma_2^{OL}, \dots, \gamma_K^{OL}\}$, that player sets

$$u_1 = \gamma_1^{OL}(x_1), \quad u_2 = \gamma_2^{OL}(x_1), \quad \dots, \quad u_K = \gamma_K^{OL}(x_1);$$

and when P_2 uses an open-loop policy $\sigma^{OL} := \{\sigma_1^{OL}, \sigma_2^{OL}, \dots, \sigma_K^{OL}\}$, that player sets

$$d_1 = \sigma_1^{OL}(x_1), \quad d_2 = \sigma_2^{OL}(x_1), \quad \dots, \quad d_K = \sigma_K^{OL}(x_1);$$

where you can think this as the set of actions given the initial condition x_1 for P_1 and P_2 , respectively.

In (perfect) *state-feedback (FB)* games, the players have perfect information and know exactly the state x_k of the game at the entry of the current stage, which can allow the player to use this information to help determine their actions u_k and d_k at that stage. However, they must choose their actions without knowing each other's choice. For a state-feedback game, the policies can be represented as functions of the current state:

when player P_1 uses a state-feedback policy $\gamma^{FB} := \{\gamma_1^{FB}, \gamma_2^{FB}, \dots, \gamma_K^{FB}\}$, that player sets

$$u_1 = \gamma_1^{FB}(x_1), \quad u_2 = \gamma_2^{FB}(x_2), \quad \dots, \quad u_K = \gamma_K^{FB}(x_K);$$

and when P_2 uses a state-feedback policy $\sigma^{FB} := \{\sigma_1^{FB}, \sigma_2^{FB}, \dots, \sigma_K^{FB}\}$, that player sets

$$d_1 = \sigma_1^{FB}(x_1), \quad d_2 = \sigma_2^{FB}(x_2), \quad \dots, \quad d_K = \sigma_K^{FB}(x_K);$$

B. Continuous-time differential games

Dynamic games are often also formulated in continuous time, which mean that

1. the state $x(t)$ varies continuously with time on a given interval $t \in [0, T]$, and
2. the players continuously select actions $u(t)$ and $d(t)$ on $[0, T]$, which determines the state's evolution.

When the state $x(t)$ is an n -vector of real numbers whose evolution is determined by differential equations, the game is called *differential game*. We considered here differential games with *dynamics* of the form

$$\dot{x}(t) = f(t, x(t), u(t), d(t)), \quad \forall t \in [0, T], \quad (22)$$

where,

- $\dot{x}(t)$ is the state derivative,
- f is the dynamic function or game dynamics,
- t is time,
- $x(t)$ is the current state,
- $u(t)$ is P_1 's action at time t ,
- $d(t)$ is P_2 's action at time t .

The outcome J_i for which each player P_i , $i \in \{1, 2\}$ wants to *minimize* a cost, can be written in the form

$$J_i := \int_0^T g_i(t, x(t), u(t), d(t))dt + q_i(x(T)), \quad (23)$$

where,

- $\int_0^T g_i(t, x(t), u(t), d(t))dt$ is *cost along trajectory*,
- $q_i(x(T))$ is the *final cost*.

Of course, we will also consider *open-loop* policies of the form

$$u(t) = \gamma^{OL}(t, x(0)), \quad d(t) = \sigma^{OL}(t, x(0)), \quad \forall t \in [0, T], \quad (24)$$

and (perfect) *state-feedback* policies of the form

$$u(t) = \gamma^{FB}(t, x(t)), \quad d(t) = \sigma^{FB}(t, x(t)), \quad \forall t \in [0, T]. \quad (25)$$

Note, when $T = \infty$, we have an *infinite horizon game* and in this case the final cost term of eq. (23) is generally absent.

C. Open vs. closed-loop: zebra in the lake

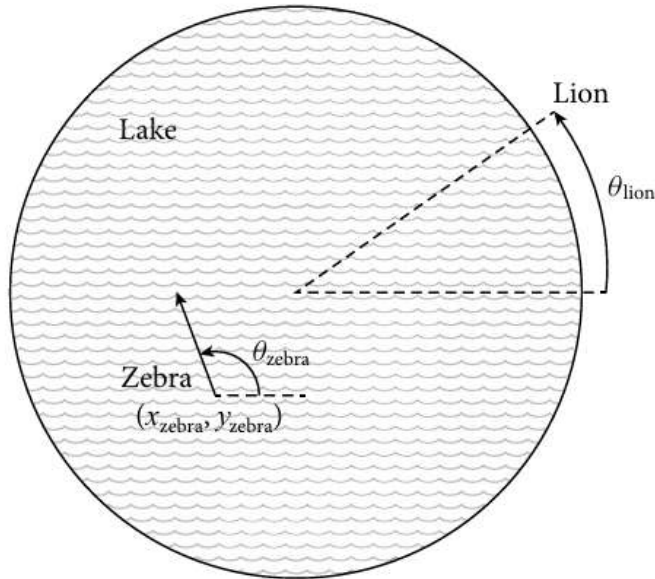


Figure 2: The zebra in the lake game from [2]

Here we want to demonstrate an example from [2], the *Zebra in the lake*⁶. The **rule of the game** for the two players:

1. The player P_1 is a *zebra* that swims in a circular lake (radius of R) with a maximum speed of v_{zebra} .
2. The player P_2 is a hydrophobic *lion* that runs along the perimeter of the lake with a maximum speed of $v_{lion} > v_{zebra}$.

⁶ This game is also known as the *lady in the lake* where a lady is trapped in the lake and a man is trying to catch her when she gets to the shore.

The **objective** of the players are:

1. The zebra wants to get to the shore of the lake without being caught coming out of water by the lion.
2. The lion wants to be at the precise position where the zebra leaves the water.

Policies: In this game it is assumed that each player constantly sees the other and can react instantaneously to the current position/velocity of the other player. This game only makes sense in closed loop if the lion can see the zebra and uses this to decide where to run, because otherwise the lion has no chance of ever catching the zebra.

Let (x_{zebra}, y_{zebra}) be the position of the zebra and θ_{zebra} be the orientation, then we have

$$\dot{x}_{zebra} = v_{zebra} \cos \theta_{zebra}, \quad \dot{y}_{zebra} = v_{zebra} \sin \theta_{zebra}, \quad \theta_{zebra} \in [0, 2\pi) \quad (26)$$

and let θ_{lion} and ω_{lion} be the angular position and angular velocity of the lion, respectively, ($\dot{\theta} = \omega$ and $v = r * \omega$) recalled from physics) then we have

$$\dot{\theta}_{lion} = \omega_{lion}, \quad \omega_{lion} \in \left[\frac{-v_{lion}}{R}, \frac{+v_{lion}}{R} \right]. \quad (27)$$

Defining a state vector

$$x(t) := [x_{zebra}(t), y_{zebra}(t), \theta_{lion}(t)]^T,$$

and the eq. (26) – (27) can be rewritten as in eq. (23) with the actions of the players

$$u(t) = \theta_{zebra} \in [0, 2\pi), \quad d(t) = \left[\frac{-v_{lion}}{R}, \frac{+v_{lion}}{R} \right].$$

If we assume that the zebra wants to escape as soon as possible without being caught, the zebra's cost is of the form

$$J_1 = \begin{cases} T_{exit}, & \text{zebra exits the lake safely at time } T_{exit} \\ +\infty, & \text{zebra get caught when it exits.} \end{cases}$$

This is a zero-sum game and therefore the lion wants to maximize J_1 ; or minimize $J_2 := -J_1$.

A common trick that is used to write such a cost in an integral form such as eq. (23) is to freeze the state when the zebra reaches the shore by

$$\begin{bmatrix} \dot{x}_{zebra} \\ \dot{y}_{zebra} \\ \dot{\theta}_{lion} \end{bmatrix} = \begin{cases} \begin{bmatrix} v_{zebra} \cos \theta_{zebra} \\ v_{zebra} \sin \theta_{zebra} \\ \omega_{lion} \end{bmatrix}, & x_{zebra}^2 + y_{zebra}^2 < R^2 \\ \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, & x_{zebra}^2 + y_{zebra}^2 = R^2, \end{cases}$$

and then defining

$$J_1 := \int_0^\infty g_1(x_{zebra}, y_{zebra}, \theta_{lion}) dt$$

where

$$g_1(x_{zebra}, y_{zebra}, \theta_{lion}) = \begin{cases} 1, & x_{zebra}^2 + y_{zebra}^2 < R^2 \\ 1, & x_{zebra} = R \cos \theta_{lion}, y_{zebra} = R \sin \theta_{lion}, \text{ zebra is caught} \\ 0, & \text{otherwise (zebra reaches shore away from lion).} \end{cases}$$

This game serves well in the context of state-feedback policies because the lion basically has no chance of capturing the zebra unless the lion can see it.

D. Differential games with variable termination time

Sometime, you want to consider the termination time, the outcome J_i for which each player P_i , $i \in \{1, 2\}$ wants to minimize a cost, can be written in the form

$$J_i := \int_0^{T_{end}} g_i(t, x(t), u(t), d(t)) dt + q_i(T_{end}, x(T_{end})), \quad (28)$$

where T_{end} is the first time at which the state $x(t)$ enters a closed set $\chi_{end} \subset \mathbb{R}^n$ or $T_{end} = +\infty$ in the case of $x(t)$ fails to enter χ_{end} . By the way, χ_{end} can be thought of as the set of states at which the game terminates where the evolution of $x(t)$ is inappropriate after this time. The states in the set χ_{end} are often called the *game-over* states.

Now continuing on with the *Zebra in the lake* example which could also be rewritten as a differential game with dynamics

$$\begin{bmatrix} \dot{x}_{zebra} \\ \dot{y}_{zebra} \\ \dot{\theta}_{lion} \end{bmatrix} = \begin{bmatrix} v_{zebra} \cos \theta_{zebra} \\ v_{zebra} \sin \theta_{zebra} \\ \omega_{lion} \end{bmatrix}, \quad \theta_{zebra} \in [0, 2\pi), \quad \omega_{lion} \in \left[\frac{-v_{lion}}{R}, \frac{+v_{lion}}{R} \right]$$

and a cost

$$J_1 = \int_0^{T_{end}} dt + q_1(x(T_{end})),$$

(Note, since P_1 wants to minimize J_1 , then here we made the *stage cost* to be 1 so we can penalize the *cost along trajectory* or time⁷) where T_{end} is the first time at which the state $x(t)$ enters a closed set

$$\chi_{end} := \{(x_{zebra}, y_{zebra}, \theta_{lion}) \in \mathbb{R}^3 : x_{zebra}^2 + y_{zebra}^2 \geq R^2\}$$

of “safe” configuration for the zebra to reach the shore and the final cost

$$q(x) := \begin{cases} 0, & \text{if } (x_{zebra}, y_{zebra}) \neq (R \cos \theta_{lion}, R \sin \theta_{lion}) \\ \infty, & \text{otherwise} \end{cases}$$

greatly penalizes the zebra the minimizer for being caught.

ONE-PLAYER DISCRETE-TIME GAMES

So far we haven’t focused on optimization. Now we want to focus on one-player discrete-time dynamic games, i.e., the optimal control of a discrete-time dynamical system. For one-player dynamic games, the dy

$$x_{k+1} = f_k(x_k, u_k), \quad \forall k \in \{1, 2, \dots, K\}, \quad (29)$$

where,

- x_{k+1} is the state at stage $k+1$,
- $f_k(\cdot)$ is a dynamic function f that gives the *dynamic* at stage k ,
- x_k denotes the state at k th stage,
- u_k denotes the action of player P_1 at the k th stage,

with initial state x_1 in the state space $\mathcal{X}$. At each iterative k , the action u_k is required to belong to a given action space U_k . Meaning, $u_k \in U_k$ where U_k is the set of feasible action at iterative k . Here, we assume this is a *finite horizon game* ($K < \infty$) with stage-additive costs of the form

$$J := \sum_{k=1}^K g_k(x_k, u_k). \quad (30)$$

When all g_k are equal to zero, except for the last g_K , the game is said to have a *terminal cost*. The only payer would minimize using either an open-loop policy

$$u_k = \gamma_k^{\text{OL}}(x_1), \quad \forall k \in \{1, 2, \dots, K\},$$

or a state feedback policy

$$u_k = \gamma_k^{\text{FB}}(x_k), \quad \forall k \in \{1, 2, \dots, K\}.$$

⁷ Can think of *penalizing time* since the zebra wants to escape as soon as possible; the longer it takes, the larger the penalty. Conversely, the lion wants to maximize and it is more rewarding to the lion as the terminal time reaches infinity. Similarly, the *final cost* is considered as a reward or punishment depending on the state of the game; small penalty if escaped or large penalty if captured, respectively.

A. Discrete-time cost-to-go

Suppose that the player is at some state x at stage ℓ , i.e., x_ℓ . This state may or may not be the optimal place to be at this stage, but for some reason the player would like to estimate the cost if playing optimally from this point on, in a sense to minimize the cost suffered in the remaining stages. This situation defines the *cost-to-go from $x \in \mathcal{X}$ at time $\ell \in \{1, 2, \dots, K\}$* by

$$V_\ell(x) := \inf_{u_\ell \in \mathcal{U}_\ell, u_{\ell+1} \in \mathcal{U}_{\ell+1}, \dots, u_K \in \mathcal{U}_K} \sum_{k=\ell}^K g_k(x_k, u_k), \quad \forall x \in \mathcal{X}, \quad (31)$$

with the sequence $\{x_k \in X : k = \ell, \ell + 1, \ell + 2, \dots, K\}$ starting at $x_\ell = x$ and satisfying the dynamics $x_{k+1} = f_k(x_k, u_k)$, $\forall k \in \{\ell, \ell + 1, \ell + 2, \dots, K - 1\}$. Note, the cost-to-go is a function of x and ℓ , and this function is often called the *value function* of the game/optimization.

One may say that it is very reasonable to minimize the cost from the beginning. In other words, we should compute the cost-to-go $V_1(x_1)$ from the initial state x_1 at the first stage $\ell = 1$, where this will minimize the cost (30) for the dynamics (29).

This leads to two important points:

1. Given that there is no constraints on (31), it is not feasible to obtain a cost (30) lower than $V_1(x_1)$. Making $V_1(x_1)$ to be always an infimum or minimum.
2. Assuming the game start with just the knowledge x_1 , if the infimum in (31) was resulted and calculated before the game starts from some specific sequences of actions (all are optimal actions of the player)

$$u_1^* \in U_1, \quad u_2^* \in U_2, \quad \dots, \quad u_K^* \in U_K$$

then this sequence of actions provides an optimal *open-loop* policy γ^{OL} :

$$\gamma_1^{OL}(x_1) := u_1^*, \quad \gamma_2^{OL}(x_1) := u_2^*, \quad \dots, \quad \gamma_K^{OL}(x_1) := u_K^*.$$

This leads to $V_1(x_1)$ equal to the smallest values that can be achieved for (29).

B. Discrete-time Dynamic Programming

In *digital-signal-processing* (DSP), a system is recursive if it depends on the past output value of the system itself. Dynamic programming is a recursive technique that is known to be capable of efficiently calculating the cost-to-go.

To get started, first let us consider the last stage K , the cost-to-go $V_K(x)$ is simply the minimum of the *final cost*, $g_K(x_K, u_K)$ over the possible actions u_K for a game that starts with $x_K = x$, and can be expressed as

$$V_K(x) = \inf_{u_K \in \mathcal{U}_K} g_K(x, u_K), \quad \forall x \in \mathcal{X}. \quad (32)$$

By the way, when $g_K(\cdot)$ is continuously differentiable, the optimization function in (31) can be solved using calculus,

$$\frac{dg_K(x_K, u_K)}{du_K} = 0$$

and giving us the global minima on a set U_K , where the derivative is zero (or at the boundary of U_K since we are dealing with infima). Therefore, for each state x , we can compute $V_K(x)$ by solving a single-parameter optimization⁸ over the set U_K .

Now, instead of the last stage, we want to consider the cost-to-go $V_\ell(x)$ at some state x at stage ℓ . For the previous stage $\ell < K$, we can rewrite (31) as

⁸ *Single-parameter optimization* refers to optimization problems where there is only one decision variable (one parameter) to be chosen in order to minimize or maximize an objective function. For example, $\min_x \{f(x)\}$, where x belongs to a set of real numbers and the optimization problem is subject to possibly being bound to $[a, b]$, where x is the single parameter (a scalar) and $f(x)$ is the objective function.

$$\begin{aligned}
V_\ell(x) &:= \inf_{u_\ell \in \mathcal{U}_\ell, \dots, u_K \in \mathcal{U}_K} \sum_{k=\ell}^K g_k(x_k, u_k) \\
&= \inf_{u_\ell \in \mathcal{U}_\ell, \dots, u_K \in \mathcal{U}_K} \left(\underbrace{g_\ell(x, u_\ell)}_{\text{independent of } u_{\ell+1}, \dots, u_K} + \underbrace{\sum_{k=\ell+1}^K g_k(x_k, u_k)}_{\text{dependent on all } u_\ell, \dots, u_K} \right) \\
&= \inf_{u_\ell \in \mathcal{U}_\ell} \left(g_\ell(x, u_\ell) + \inf_{u_{\ell+1} \in \mathcal{U}_{\ell+1}, \dots, u_K \in \mathcal{U}_K} \sum_{k=\ell+1}^K g_k(x_k, u_k) \right), \tag{33}
\end{aligned}$$

where in the first equality we used the fact that we must set $x_\ell = x$ to compute $V_\ell(x)$ and in the second equality we used the fact that $g_\ell(x, u_\ell)$ does not depend on $u_{\ell+1}, \dots, u_K$. Notice that

$$\inf_{u_{\ell+1} \in \mathcal{U}_{\ell+1}, \dots, u_K \in \mathcal{U}_K} \sum_{k=\ell+1}^K g_k(x_k, u_k)$$

is actually the minimum cost for a game starting at stage $\ell+1$ with the state $x_{\ell+1} = f_\ell(x, u_\ell)$ where it depends on the previous state and previous action, this can get us the cost-to-go $V_{\ell+1}(f_\ell(x, u_\ell))$. Therefore,

$$V_\ell(x) = \inf_{u_\ell \in \mathcal{U}_\ell} \left(g_\ell(x, u_\ell) + V_{\ell+1}(f_\ell(x, u_\ell)) \right), \quad \forall x \in \mathcal{X}, \quad \ell \in \{1, 2, \dots, K-1\}. \tag{34}$$

This shows that if we know the value function $V_{\ell+1}(\cdot)$, then we can compute each $V_\ell(x)$ by solving a single-parameter optimization over the set $\mathcal{U}_\ell$. Moreover, this optimization produces the optimal action u_ℓ^* when the state is at x_ℓ .

It is also allowed to define

$$V_{K+1}(x) = 0, \quad \forall x \in \mathcal{X},$$

which allows us to rewrite both (32) & (34) using the formula:

$$V_\ell(x) = \inf_{u_\ell \in \mathcal{U}_\ell} \left(g_\ell(x, u_\ell) + V_{\ell+1}(f_\ell(x, u_\ell)) \right), \quad \forall x \in \mathcal{X}, \tag{35}$$

that is **now valid** $\forall \ell \in \{1, 2, \dots, K\}$.

Open-loop policy: Out of curiosity, for the particular case of stage $\ell=1$ and state $x = x_1$ and when the infima in (35) are actually the minima, the points at which these infima are achieved can be used to construct an open-loop policy. Specifically from [2], we can obtain:

- u_1^* from (34) with $\ell=1$ and $x = x_1$, leading to $x_2^* = f_1(x_1, u_1^*)$;
- u_2^* from (34) with $\ell=2$ and $x = x_2^*$, leading to $x_3^* = f_2(x_2^*, u_2^*)$;
- u_3^* from (34) with $\ell=3$ and $x = x_3^*$, and so on.

Of course, this shows that we need follow the procedure below, to be able to compute the open loop policy γ^{OL} that minimizes the cost (30) for the dynamics (29):

1. First, we need to calculate the cost-to-go using a *backward iteration* starting from $\ell=K$ and proceeding backward in time until we have reached $\ell=1$ using

$$V_{K+1}(x) = 0, \quad V_\ell(x) = \inf_{u_\ell \in \mathcal{U}_\ell} (g_\ell(x, u_\ell) + V_{\ell+1}(f_\ell(x, u_\ell))), \quad \forall x \in \mathcal{X}. \quad (36)$$

Notice, in order to accomplish backward iteration, we need to calculate $V_\ell(x)$ for every possible value of the state x at stage ℓ . Also, (36) is usually computed using software such as MATLAB since calculating by hand can be time-consuming.

2. Second, calculate the sequence of actions

$$u_1^* \in \mathcal{U}_1, u_2^* \in \mathcal{U}_1, \dots, u_K^* \in \mathcal{U}_K,$$

that minimizes $V_1(x_1)$ using a *forward iteration* starting from $k=1$ and end at $k=K$:

$$x_1^* = x_1, \quad u_k^* = \arg \min_{u_k \in \mathcal{U}_k} \underbrace{(g_k(x_k^*, u_k) + V_{k+1}(f_k(x_k^*, u_k)))}_{\text{computed using the precomputed states } x_k^*}, \quad x_{k+1}^* = f_k(x_k^*, u_k^*). \quad (37)$$

By the way, (37) is calculated under the assumption that the infimum of $g_\ell(x_\ell^*, u_\ell) + V_{\ell+1}(f_\ell(x_\ell^*, u_\ell))$ is feasible at some point $u_k \in U_k$. This calculation of course fails if it is not feasible. Also, when the infimum is found at multiple points, we are free to choose any one of them to be used in (37).

3. Finally, the optimal open-loop policy γ^{OL} is given by

$$\gamma_1^{OL}(x_1) := u_1^*, \quad \gamma_2^{OL}(x_1) := u_2^*, \quad \dots, \quad \gamma_K^{OL}(x_1) := u_K^*. \quad (38)$$

Notice that all the optimal states x_k^* and optimal actions u_k^* are pre-calculated in (37) and depend on the only information available which is the initial state x_1 , making (38) an open-loop policy.

State-feedback policy: If we are given or calculated the optimal open-loop γ^{OL} from (38) which select the optimal actions

$$u_k = \gamma_k^{OL}(x_1) := u_k^*, \quad \forall k \in \{1, 2, \dots, K\}.$$

and the pre-calculated stated x_k^* defined by (37) which matches x_k by whom is measured during the game, then we can get a closed estimate minimum value $V_1(x_1)$ for the cost (30), if we were using a state-feedback policy γ^{FB} defined by

$$\gamma_k^{FB}(x_k) := \arg \min_{u_k \in \mathcal{U}_k} \underbrace{(g_k(x_k, u_k) + V_{k+1}(f_k(x_k, u_k)))}_{\text{computed using the measured state } x_k}, \quad \forall k \in \{1, 2, \dots, K\}. \quad (39)$$

By the way, (39) is calculated under the assumption that the infimum of $g_\ell(x_\ell, u_\ell) + V_{\ell+1}(f_\ell(x_\ell, u_\ell))$ is feasible at some point $u_k \in U_k$. This calculation of course fails if it is not feasible. Also, when the infimum is found at multiple points, we are free to choose any one of them to be used in (39). Now, after all the minimum of $g_\ell(x_\ell, u_\ell) + V_{\ell+1}(f_\ell(x_\ell, u_\ell))$ is obtained for some $u_k \in U_k$, then we got our optimal state-feedback policy is (39) since it is feasible to obtain a value for the cost (30) lower than $V_1(x_1)$.

C. Linear quadratic dynamic games

I want to mention discrete-time *linear quadratic one-player games* from [2] because their linear dynamic form is related to the *state-space representation* of a discrete-time system. The discrete-time linear quadratic one-player games are characterized by linear dynamics of the form

$$x_{k+1} = \underbrace{Ax_k + Bu_k}_{f_k(x_k, u_k)}, \quad x \in \mathbb{R}^n, \quad u \in \mathbb{R}^{n_u}, \quad k \in \{1, 2, \dots, K\}$$

and a stage-additive quadratic cost of the form

$$J := \sum_{k=1}^K \underbrace{\left(\|y_k\|^2 + u_k' R u_k \right)}_{g_k(x_k, u_k)} = \sum_{k=1}^K \underbrace{\left(x_k' C' C x_k + u_k' R u_k \right)}_{g_k(x_k, u_k)},$$

where

$$y_k = Cx_k, \quad \forall k \in \{1, 2, \dots, K\}.$$

Note, the notation (') means *transposed*. According to [2], this cost function allows the only player here to make the y_k where $k \in \{1, 2, \dots, K\}$ small without “spending” much effort in their action u_k . By the way, the matrix R is a symmetric positive matrix that is used to map the units of u_k into the units of y_k .

If we need to find the optimal state-feedback policy, then we need to use *backward* recursion or a *difference Riccati equation* as shown in (40):

$$P_{K+1} = 0, \tag{40.a}$$

$$P_k = C' C + A' P_{k+1} A - A' P_{k+1} B (R + B' P_{k+1} B)^{-1} B' P_{k+1} A, \quad \forall k \in \{1, 2, \dots, K\}, \tag{40.b}$$

and that

$$R + B' P_{k+1} B \geq 0, \quad \forall k \in \{1, 2, \dots, K\}. \tag{41}$$

Then the optimal state feedback policy is

$$\gamma_k^{FB}(x_k) = -(R + B' P_{k+1} B)^{-1} B' P_{k+1} A x_k, \quad \forall k \in \{1, 2, \dots, K\}$$

All you need to do is calculate matrix P_k and plug in other matrices and states at stage k , then you will be able to find the optimal state-feedback policy for the linear quadratic one-player, leading to an optimal cost equal to $x_1' P_1 x_1$.

The matrix *difference Riccati equation* is actually used to obtain the feedback gain values for a missile/target engagement model, where it helps guide the missile.

ONE-PLAYER CONTINUOUS-TIME DIFFERENTIAL GAMES

Considered here one-player differential games with dynamics of the form

$$\dot{x}(t) = f(t, x(t), u(t)), \quad \forall t \in [0, T], \tag{42}$$

where,

- $\dot{x}(t)$ is the state derivative,
- f is the dynamic function or game dynamics,
- t is time,
- $x(t)$ is the current state,
- $u(t)$ is P_1 's action at time t ,

with state $x(t) \in \mathbb{R}^n$ and the initial condition $x(0) = x_0$. Note, the action $u(t)$ must belong to a given action space U for every time $t \in [0, T]$. Here, we assume a finite horizon ($T < \infty$) integral cost of the form

$$J := \int_0^T g_i(t, x(t), u(t)) dt + q(x(T)), \tag{43}$$

where,

- $\int_0^T g(t, x(t), u(t))dt$ is *cost along trajectory*,
- $q(x(T))$ is the *final cost*.

By the way, the only payer would minimize using either an open-loop policy

$$u(t) = \gamma^{\text{OL}}(t, x_0), \quad \forall t \in [0, T],$$

or a state-feedback policy

$$u(t) = \gamma^{\text{FB}}(t, x(t)), \quad \forall t \in [0, T].$$

A. Continuous-time cost-to-go

The definition of cost-to-go for a differential game is just like for the discrete game except now the time is continuous. Suppose that the player is at some state x at time τ , i.e., x_τ . This state may or may not be the optimal place to be at this stage, but for some reason the player would like to estimate the cost if playing optimally from this point on, in a sense to minimize the cost suffered in the remaining time. This situation defines the *cost-to-go from x at time τ* by

$$V(\tau, x_\tau) := \inf_{u(t) \in \mathcal{U}, \forall t \in [\tau, T]} \int_\tau^T g(t, x(t), u(t))dt + q(x(T)) \quad (44)$$

with the state $x(t)$, $t \in [\tau, T]$ starting at $x(\tau) = x_\tau$ and satisfying the dynamics $\dot{x}(t) = f(t, x(t), u(t))$, $\forall t \in [\tau, T]$. Note, the cost-to-go is a function of x and τ , and this function is often called the *value function* of the game/optimization.

One may say that it is very reasonable to minimize the cost from the beginning. In other words, we should compute the cost-to-go $V(0, x_0)$ from the initial state x_0 at time $\tau=0$, where this will minimize the cost (43) for the dynamics (42). This leads to two important points:

1. Given that there is no constraints on (44), it is not feasible to obtain a cost (43) lower than $V(0, x_0)$. Making $V(0, x_0)$ to be always an infimum or minimum.
2. Assuming the game start with just the knowledge x_1 , if the infimum in (44) was resulted and calculated before the game starts from some specific signal of actions (all are optimal actions of the player)

$$u^*(t) \in \mathcal{U}, \quad t \in [\tau, T]$$

then this signal of actions provides an optimal *open-loop* policy γ^{OL} :

$$\gamma_1^{\text{OL}}(t, x_0) := u^*(t), \quad \forall t \in [\tau, T].$$

This leads to $V(0, x_0)$ equal to the smallest values that can be achieved for (43). Note, this would not be true if there were stochastic events involved.

B. Continuous-time Dynamic Programming

Like before, first let us consider the final time T , the cost-to-go $V(T, x_T)$ is simply the *final cost* $V(T, x_T) = q(x(T)) = q(x_T)$, for a game that starts (and ends) at time $x(T) = x_T$ because the integral term in (44) would cancel out due $\tau = T$.

Now, instead of the final time, we want to consider the cost-to-go $V(\tau, x_\tau)$ at some state x at time τ . For some time $\tau < T$ and choose a small positive constant h such that $\tau + h$ is still smaller than T , we can rewrite (44) after doing some math as

$$V(\tau, x_\tau) = \inf_{u(t) \in \mathcal{U}, \forall t \in [\tau, \tau+h]} \left(\int_\tau^{\tau+h} g(t, x(t), u(t))dt + V(\tau+h, x(\tau+h)) \right).$$

Then we can subtract $V(\tau, x_\tau) = V(\tau, x(\tau))$ on both sides and dividing both sides by $h>0$, getting us

$$0 = \inf_{u(t) \in \mathcal{U}, \forall t \in [\tau, \tau+h)} \left(\frac{1}{h} \int_{\tau}^{\tau+h} g(t, x(t), u(t)) dt + \frac{V(\tau+h, x(\tau+h)) - V(\tau, x(\tau))}{h} \right). \quad (45)$$

Also, the left-hand-side must equal to zero for every $h \in (0, T - \tau)$, where as the limit of the right-hand-side as $h \rightarrow 0$ must be zero. Here, we are under the assumption that the all limit of the infimum doesn't change, which resulted with the infamous *Hamilton-Jacobi-Bellman (HJB)* equation:

$$0 = \inf_{u \in \mathcal{U}} \left(g(\tau, x, u) + \frac{\partial V(\tau, x)}{\partial \tau} + \frac{\partial V(\tau, x)}{\partial x} f(\tau, x, u) \right), \quad \forall \tau \in [0, T], x \in \mathbb{R}^n. \quad (46)$$

It turns out that the HJB equation is very useful to compute the cost-to-go. Also, any differential function $V(\tau, x)$ that satisfies (46) with

$$V(T, x) = q(x), \quad \forall x \in \mathbb{R}^n \quad (47)$$

is equal to the cost-to-go $V(\tau, x)$. In addition, if the infimum in (46) is always achieved at some point in U , we have that:

1. For any given x_0 , an optimal open-loop policy γ^{OL} is given by

$$\gamma^{OL}(t, x_0) := u^*(t), \quad \forall t \in [0, T]$$

where

$$\begin{aligned} u^*(t) &= \arg \min_{u \in \mathcal{U}} g(t, x^*(t), u) + \frac{\partial V(t, x^*(t))}{\partial x} f(t, x^*(t), u) \\ \dot{x}^*(t) &= f(t, x^*(t), u^*(t)), \quad \forall t \in [0, T], \quad x^*(0) = x_0. \end{aligned}$$

2. An optimal state-feedback policy γ^{FB} is given by

$$\gamma^{FB}(t, x(t)) := \arg \min_{u \in \mathcal{U}} g(t, x(t), u) + \frac{\partial V(t, x(t))}{\partial x} f(t, x(t), u), \quad \forall t \in [0, T]. \quad (48)$$

Note, either of the optimal policies lead to an optimal cost equal to $V(0, x_0)$.

C. Linear quadratic dynamic games

I want to mention continuous-time *linear quadratic one-player games* from [2] because their linear dynamic form is related to the *state-space representation* of a continuous-time system for control systems. The continuous-time linear quadratic one-player games are characterized by linear dynamics of the form

$$\dot{x} = \underbrace{Ax(t) + Bu(t)}_{f(t, x(t), u(t))}, \quad x \in \mathbb{R}^n, \quad u \in \mathbb{R}^{n_u}, \quad t \in [0, T]$$

and an integral quadratic cost of the form

$$J := \int_0^T \underbrace{\left(\|y(t)\|^2 + u(t)' R u(t) \right)}_{g(t, x(t), u(t))} dt + \underbrace{x'(T) P_T x(T)}_{q(x(T))},$$

where

$$y(t) = Cx(t), \quad \forall t \in [0, T].$$

According to [2], this cost function allows the only player here to make the $y(t)$ over the time interval $[0, T]$ small without “spending” much effort in their action $u(t)$. By the way, the matrix R is a symmetric positive matrix that is used to map the units of $u(t)$ into the units of $y(t)$.

Now a simple optimal control system example can be used here, such that we want to optimize the output temperature of an air-conditioner unit. The player can be the user and the action be the inputted signal from a remote control. The state can be the voltage/current from the electrical components and the cost can be something like temperature error or energy lost. Here we

can use dynamic games to reduce our losses. If this was a two-player game, the second player can be the inner or external noise causing temperature errors.

Conclusion

We can see that zero-sum games are a subset of non-cooperative games because a player's gain is another player's loss. The rope-pulling helps us physically imagine two-players being uncooperative where each of their gains canceled each other out. In a zero-sum game, a saddle point is also referred to as *pure-strategy Nash equilibrium* because none of the players are not willing to deviate from the pair strategy or equilibrium pair, and this is the best response against each other, which will always be chosen as the main strategy for both players. Advancing from single-stage games to multi-stage games allows us to reach dynamic games, whereas differential games are essentially dynamic games except it is in continuous time. In dynamic games, we were introduced to information structures to open-loop and state-feedback policies, which then leads to our example *zebra in the lake* game. The *zebra in the lake* game, which is classified as a *pursuit-evasion* game, was able to demonstrate the state-feedback policies because the lion basically has no chance of capturing the zebra unless the lion can see it. In other words, if we are looking at the game as an open-loop game, the lion is not able to see the zebra and has no feedback that helps make a decision on the direction that allows the lion to capture it. Afterward, we briefly introduced one-player dynamic games and one-player differential games, where we discussed value functions, dynamic programming, and optimal policies. In addition, we were also given linear quadratic dynamic games due to its correlation with state-space representation in control systems. Although there are many other topics about optimal control and differential game theory that were not mentioned, such as H-infinity or the Hamilton-Jacobi-Bellman-Isaac equation, the topics that are studied here can serve as a foundation to more advanced topics of dynamic or differential games.

Reference

1. F. A. Faruqi, P. H. Zipfel, J. P. Cooper, and A. Seabridge, *Differential Game Theory with Applications to Missiles and Autonomous Systems Guidance*. Hoboken, NJ, USA: Wiley, 2019.
2. J. P. Hespanha, *Noncooperative Game Theory: An Introduction for Engineers and Computer Scientists*. Princeton, NJ, USA: Princeton Univ. Press, 2017.
3. E. Mendelson and D. Zwillinger, *Introducing Game Theory and Its Applications*. Boca Raton, FL, USA: Chapman & Hall/CRC Press, 2016.